

Function

정의부터 다양한 형태까지



20212105 정보통신공학과 강은현

목차

01 함수와 함수 리터럴

02 함수 정의

03 함수 호출

04 참조에 의한 전달

05 다양한 함수의 형태

01

SECTION

함수와 함수 리터럴

하나의 실행 단위로 묶은 코드

함수란

함수는 코드를 묶은 실행 단위임

- 1 일련의 과정을 코드 블록으로 감싼 하나의 실행 단위임
- 2 객체 타입의 값이며, 함수 이름으로 구별함
- 3 재사용성을 높여 유지보수에 유리함

함수 리터럴

함수 리터럴은 평가되면 객체가 됨

```
function add(x, y) {  
  return x + y;  
};
```

일반 객체와 달리 호출할 수 있는 객체임

함수 리터럴

함수 리터럴의 구성 요소

```
function add(x, y) { return x + y; }
```

함수 이름

add · 식별자, 생략 가능

매개변수 목록

(x, y) · 0개 이상, 순서대로
할당

함수 몸체

{ ... } · 실행할 문들의 코드
블록

02

SECTION

함수 정의

실행에 필요한 것을 미리 지정하는 것

함수 정의는 실행 준비를 갖추는 것임

매개변수·실행할 문·반환값을 지정하면, 엔진이 평가해 함수 객체를 생성함

정의 방법

함수를 정의하는 네 가지 방법

1

함수 선언문

2

함수 표현식

3

Function 생성자 함수

4

화살표 함수

함수 선언문

선언문은 이름을 생략할 수 없음

```
function add(x, y) {  
    return x + y;  
}
```

함수 리터럴과 형태가 같지만, 표현식이 아닌 문임

함수 선언문

선언문의 완료 값은 undefined임

```
function add(x, y) {  
    return x + y;  
}  
  
// undefined
```

엔진은 모든 문의 실행 결과를 완료 값으로 기록함

함수 선언문

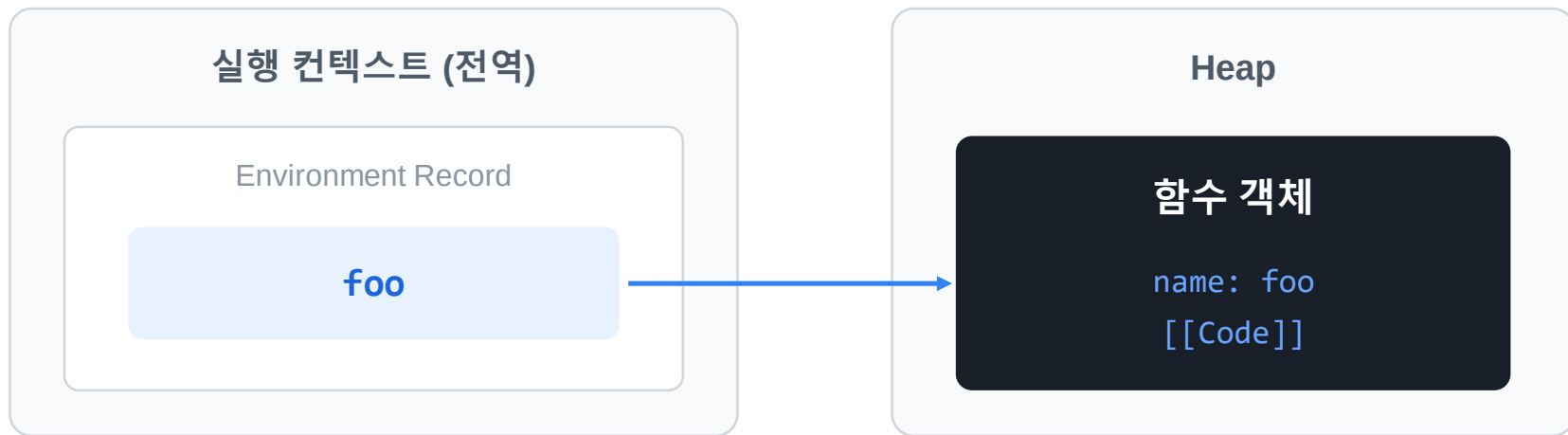
식별자 없이도 호출됨

```
function foo() {  
  console.log("foo");  
}  
  
foo();
```

엔진이 이름과 같은 식별자를 암묵적으로 생성해 바인딩함

함수 선언문

엔진이 식별자를 만들어 객체에 연결함



foo() 는 함수 이름이 아니라, 엔진이 만든 식별자를 통해 함수 객체를 호출함

함수 표현식

함수는 변수에 할당할 수 있음

```
// 함수 선언문
```

```
function add(x, y) {  
  return x + y;  
}
```

```
// 함수 표현식
```

```
let add = function (x, y) {  
  return x + y;  
};
```

함수 표현식

식별자가 생기는 방식이 다름

함수 선언문

엔진이 식별자 `add` 를 암묵 생성

함수 표현식

개발자가 변수 `add` 를 선언

함수 객체

(Heap)

두 방식 모두 함수 객체를 만들지만, 그 객체를 참조하는 식별자를 만드는 주체가 다름

같은 리터럴도 문맥에 따라 다르게 해석됨

단독으로 쓰면 선언문, 값이 필요한 위치에 쓰면 표현식으로 해석함

선언문과 표현식은 형태가 유사함

```
function add(x, y) {  
  return x + y;  
}
```

// 함수 표현식

```
let add = function (x, y) {  
  return x + y;  
}
```

리터럴 해석

단독으로 쓰면 선언문으로 해석함

```
function foo() {  
  console.log("foo");  
}  
  
foo();
```

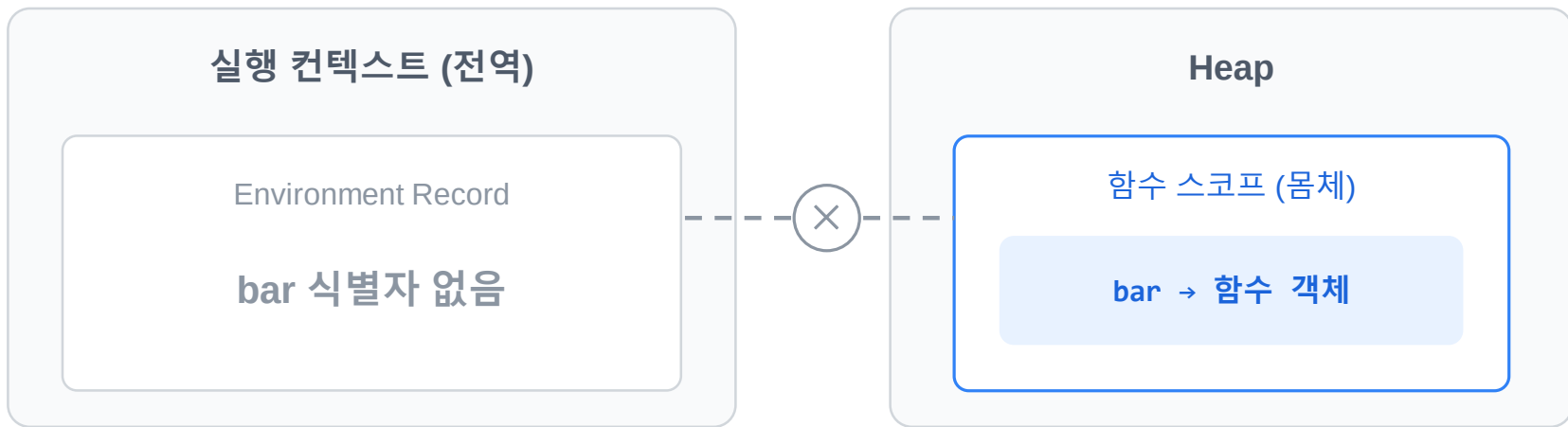
`}`이 위치에 따라 블록문·객체 리터럴로 갈리듯, 함수 리터럴도 단독 사용 시 선언문이 됨

그룹 연산자 안에서는 표현식이 됨

```
(function bar() {  
  console.log("bar");  
});  
  
bar(); // ReferenceError
```

표현식은 식별자를 만들지 않아 외부에서 부를 수 없음

함수 이름은 몸체 안에서만 유효함



함수 이름 bar 는 함수 스코프 내부 전용이라, 외부 실행 컨텍스트에서는 참조할 수 없음

이름 있는 함수 표현식도 마찬가지로임

```
let foo = function bar() {  
  console.log("foo");  
};
```

```
foo(); // 정상 호출
```

```
bar(); // ReferenceError
```

함수 이름은 호출용이 아니라 몸체 내부 전용 식별자임

함수 생성 시점과 함수 호이스팅

선언문과 표현식은 생성 시점이 다름

```
console.dir(add); // [Function: add]
console.dir(sub); // ReferenceError

function add(x, y) { return x + y; }
let sub = function (x, y) { return x - y; };
```

add는 먼저 호출해도 되지만, sub는 ReferenceError가 남

함수 생성 시점과 함수 호이스팅

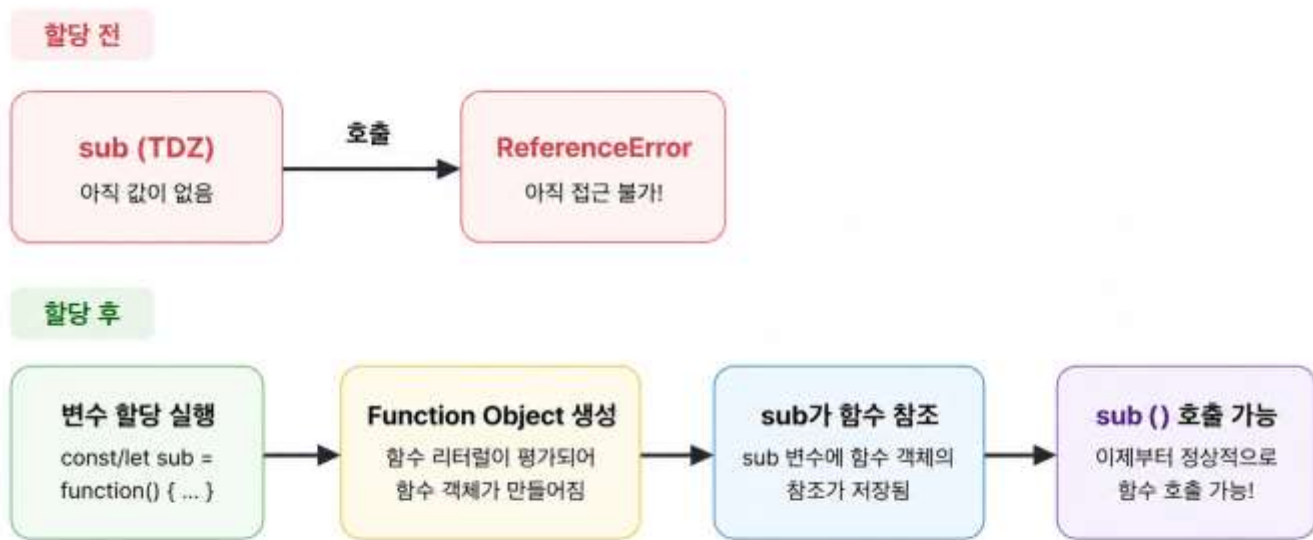
표현식은 함수가 아니라 변수만 호이스팅됨

```
let sub = function (x, y) {  
  return x - y;  
};
```

함수 선언문은 함수 객체까지 미리 생성되지만, 표현식은 변수 sub 만
먼저 만들어짐

함수 생성 시점과 함수 호이스팅

표현식은 TDZ의 영향을 받음



Function 생성자

생성자 함수는 new로 객체를 만듦

```
const arr = new Array();  
const date = new Date();
```

Array · Date · Map 등도 자바스크립트가 제공하는 생성자 함수임

Function 생성자

Function 생성자로도 만들 수 있음

```
let add = new Function('x', 'y', 'return x + y');  
  
console.log(add(2, 5)); // 7
```

매개변수와 몸체를 문자열로 전달함 (new 생략 가능)

하지만 거의 쓰지 않음

문자열을 런타임에 파싱하고, 주변 변수를 기억하지 않아
클로저를 만들지 않으며 항상 전역 스코프에서 동작함

화살표 함수

화살표 함수는 더 간결한 표현식임

```
const add = (x, y) => x + y;
```

ES6 에서 도입, 항상 함수 표현식 형태로만 작성함

화살표 함수는 동작도 간결함

1

생성자 함수로 사용할 수 없음

2

this 바인딩 방식이 다름

3

prototype 프로퍼티가 없음

4

arguments 객체를 만들지 않음

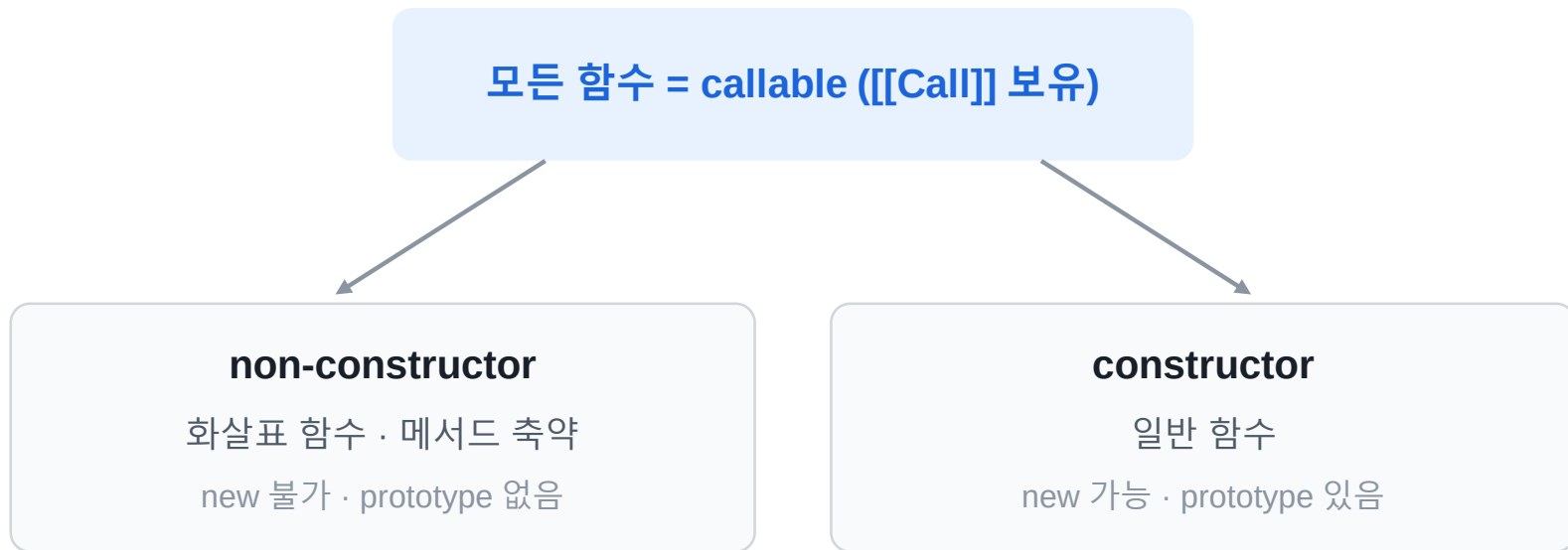
내부 메서드

함수 객체의 두 내부 메서드



모든 함수는 [[Call]] 을 갖지만, [[Construct]] 는 일부 함수만 가짐

모든 함수가 생성자인 것은 아님



화살표 함수는 non-constructor 로 생성되어 new 로 호출할 수 없음

**그래서 화살표 함수는
new 로 부를 수 없음**

non-constructor로 생성되어 `[[Construct]]`와 `prototype`이 없음

03

SECTION

함수 호출

인수를 넘기고 실행 흐름을 이동하는 것

함수 호출

함수는 소괄호로 호출함

```
function add(x, y) {  
  return x + y;  
}  
  
let result = add(1, 2);
```

인수는 매개변수에 순서대로 할당되고 몸체가 실행됨

매개변수는 몸체 안의 변수임

매개변수를 통해 전달된 인수는 개수와 타입에 제한이 없음

매개 변수와 인수

호출하면 매개 변수가 만들어짐



호출 시 매개 변수가 암묵 생성되어 `undefined` 로 초기화된 뒤, 인수가 순서대로 할당됨

매개변수와 인수

개수가 달라도 에러가 없음

```
function add(x, y) {  
  console.log(x);  
  console.log(y);  
}  
  
add(1);          // 1 undefined  
add(1, 2, 3);    // 1 2
```

부족하면 undefined, 초과된 인수는 arguments에 담김

arguments 객체

arguments는 모든 인수를 담음

```
function sum() {  
  console.log(arguments);  
}  
  
sum(1, 2, 3, "hi");  
// [Arguments] { '0': 1, '1': 2, '2': 3, '3': 'hi' }
```

호출 시 자동으로 생성되는 유사 배열 객체임

arguments vs Rest

인수를 모두 써야 하면 Rest Parameter를 씀

```
const sum = (...args) => {  
  console.log(args);  
};
```

매개변수 앞에 ...을 붙이면 인수들이 실제 배열로 담김

가장 큰 차이는 실제 배열 여부임

arguments

유사 배열 객체이며, 화살표 함수에는 존재하지 않음

Rest Parameter

실제 배열이라 배열 메서드를 바로 사용할 수 있음

반환문

return으로 결과를 반환함

```
function multiply(x, y) {  
  return x * y;  
}  
  
let result = multiply(3, 5); // 15
```

함수 호출은 표현식이라 반환값으로 평가됨

반환문

return은 함수 실행을 멈춤

```
function multiply(x, y) {  
  return x * y;  
  console.log('실행'); // 실행되지 않음  
}
```

반환문

표현식을 지정하지 않으면 undefined임

```
function foo() {  
  return;  
}
```

```
console.log(foo()); // undefined
```

반환문

반환문을 생략해도 undefined임

```
function foo() {  
  }  
  
console.log(foo()); // undefined
```

04

SECTION

참조에 의한 전달

매개변수도 값·참조 방식을 따른다

참조 전달

원시 값은 복사되어 전달됨

```
function changeVal(primitive, obj) {  
    primitive += 100;  
    obj.name = 'Kim';  
}  
  
let num = 100;  
let person = { name: '은현' };  
changeVal(num, person);
```

원시 값은 원본이 보존되지만, 객체는 원본이 바뀜

참조 전달

객체는 원본이 함께 바뀜

Before

원시 값

num: 100

primitive: 100

객체

person · obj



{ name: '은현' }

After

원시 값

num: 100

primitive: 200

객체

person · obj



{ name: 'Kim' }

원시 값은 원본이 그대로지만, 객체는 참조가 복사되어 원본이 함께 변경됨

05

SECTION

다양한 함수의 형태

상황에 맞게 골라 쓰는 함수들

즉시 실행 함수

정의와 동시에 즉시 호출함

```
(function () {  
    console.log('Hello');  
})();
```

정의와 호출을 하나의 표현식으로 작성하므로, 생성되는 즉시 한 번만 실행됨

즉시 실행 함수

익명 함수뿐 아니라 기명 함수도 씀

```
(function foo () {  
  console.log('Hello');  
})();
```

즉시 실행 함수는 보통 익명 함수를 쓰지만 기명 함수도 쓸 수 있음

즉시 실행 함수

이름이 있어도 밖에선 못 부름

```
(function foo () {  
  foo(); // 가능  
})();  
  
foo(); // ReferenceError
```

그룹 연산자 안의 기명 함수는 함수 표현식으로 평가되어, 외부에서는 다시 호출할 수 없음

즉시 실행 함수

그룹 연산자 외에도 만들 수 있음

```
(function () {})(());
```

```
(function () {}());
```

```
!function () {}();
```

```
+function () {}();
```

즉시 실행 함수

반환값과 인수도 쓸 수 있음

```
const result = (function (a, b) {  
    return a * b;  
})(3, 5);  
  
console.log(result);
```

재귀 함수

재귀 함수는 자기 자신을 호출하는 함수

```
function countdown(n) {  
  for (let i = n; i >= 0; i--) {  
    console.log(i);  
  }  
}
```

```
countdown(10);
```

재귀 함수

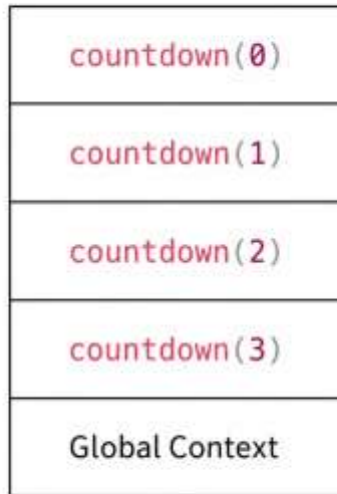
같은 동작을 재귀로도 구현함

```
function countdown(n) {  
  if (n < 0) return;  
  
  console.log(n);  
  countdown(n - 1);  
}  
  
countdown(10);
```

재귀 함수

호출마다 콜 스택에 쌓임

Top



Bottom

재귀 함수

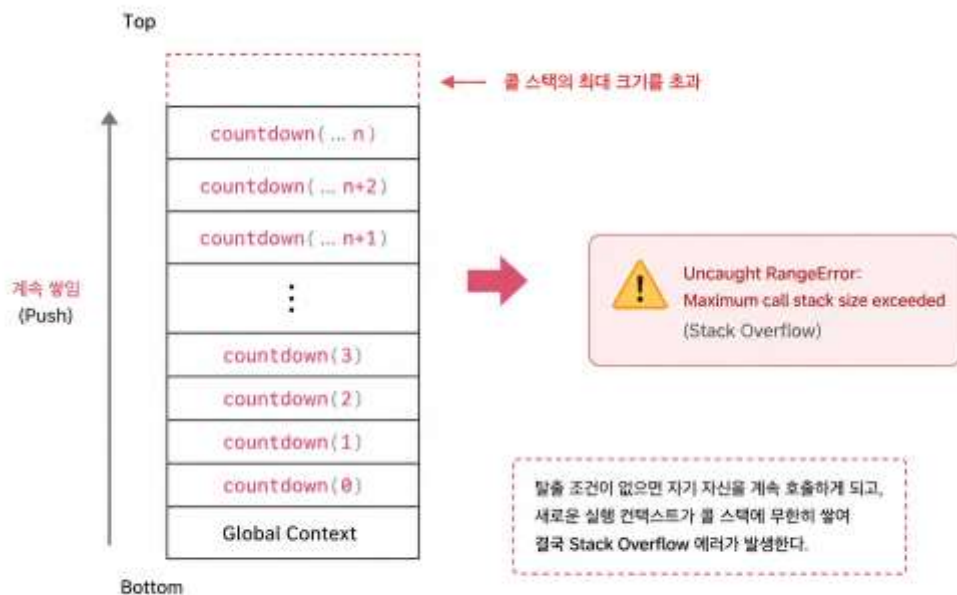
반드시 탈출 조건이 있어야 함

```
if (n < 0) return;
```

탈출 조건을 만족하면 가장 마지막에 호출된 함수부터 차례로 종료되며 콜 스택에서 제거됨

재귀 함수

탈출 조건이 없으면 Stack Overflow



재귀 vs 반복

언제 무엇을 쓸까

1. 단순 반복 → 반복문

같은 작업을 일정 횟수 반복할 때 사용

예) 1부터 5까지 출력

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

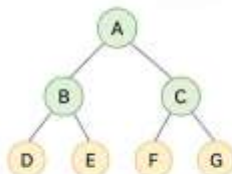


같은 작업을 계속 반복

2. 계층 구조 탐색 → 재귀 함수

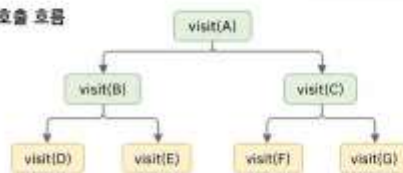
같은 처리 방식을 자식에도 반복해서 적용할 때 사용

예) 트리의 모든 노드 방문



```
function visit(node) {  
  console.log(node.value);  
  for (const child of node.children) {  
    visit(child);  
  }  
}
```

호출 흐름



- 현재 노드 방문
- 자식 노드로 이동
- 귀환 노드 방문

반복문은 같은 작업을 여러 번 수행할 때,
재귀 함수는 자식도 같은 방식으로 처리해야 하는 계층 구조를 탐색할 때 자연스럽다.

중첩 함수

함수 안에 정의한 함수임

```
function outer() {  
  let x = 1;  
  function inner() {  
    let y = 2;  
    console.log(x + y); // 3  
  }  
  inner();  
}
```

```
inner(); // ReferenceError
```

중첩 함수

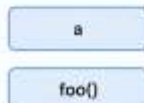
스코프는 식별자의 유효 범위임

전역 스코프
(Global Scope)

```
let a = 1;

function foo() {
  console.log(a);
}
```

Global Scope



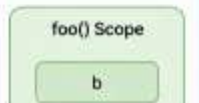
코드의 가장 바깥에 선언된 식별자
어디서든 접근 가능

함수 스코프
(Function Scope)

```
function foo() {
  let b = 2;
  console.log(b);
}

foo();
```

Global Scope



함수 내부에 선언된 식별자
함수 안에서만 접근 가능

블록 스코프
(Block Scope)

```
if (true) {
  let c = 3;
}

console.log(c); // Error
```

Global Scope



블록({}) 내부에 선언된 식별자
블록 안에서만 접근 가능

중첩 함수

안에서 못 찾으면 바깥 스코프를 탐색함

```
function outer() {  
  let x = 1;  
  function inner() {  
    let y = 2;  
    console.log(x + y); // 3  
  }  
  inner();  
}
```

매개변수로 전달되어 나중에 실행되는 함수임

콜백 함수를 전달받아 실행하는 함수를 고차 함수라고 함

콜백 함수

고차 함수가 콜백을 필요할 때마다 실행함

```
function repeat(n, callback) {  
  for (let i = 0; i < n; i++) {  
    callback(i);  
  }  
}
```

```
function log(i) {  
  console.log(i);  
}
```

```
repeat(5, log);
```


콜백 함수

함수 자체를 전달해야 함

```
repeat(5, log); // 정상적인 형태
```

```
repeat(5, log()); // 비정상적인 형태
```

인수는 호출 전에 평가되므로, `log()`가 아니라 `log` 자체를 전달해야 필요한 시점에 직접 호출할 수 있음

콜백 함수

실제로 매우 자주 쓰이는 구조임

```
setTimeout(callback, 1000);
```

```
button.addEventListener("click", callback);
```

```
array.map(callback);
```

모두 언제 콜백을 실행할지는 콜백을 전달받은 고차 함수가 결정함

콜백 함수

매개변수 n만큼 숫자를 출력함

```
function repeat(n) {  
  for (let i = 0; i < n; i++) {  
    console.log(i);  
  }  
}  
  
repeat(5);
```

repeat 함수는 숫자를 출력하는 기능에 강하게 의존하고 있음

조건이 바뀌면 새 함수가 필요함

```
function repeatOdd(n) {  
  for (let i = 0; i < n; i++) {  
    if (i % 2) console.log(i);  
  }  
}
```

반복 로직은 같은데 작업만 달라져도 매번 새 함수를 작성해야 하는 문제가 생김

콜백 함수

공통 로직은 고차 함수가, 변경은 콜백이 맡음

```
function repeat(n, callback) {  
  for (let i = 0; i < n; i++) {  
    callback(i);  
  }  
}
```

```
function logAll(i) {  
  console.log(i);  
}
```

```
repeat(5, logAll);
```

순수 함수

순수 함수는 외부에 의존하지 않음

```
function increase(n) {  
  return ++n;  
}  
  
let count = 0;  
count = increase(count); // 1  
count = increase(count); // 2
```

전달받은 값만 써서 결과가 늘 예측 가능함

비순수 함수

비순수 함수는 외부 상태를 바꿈

```
let count = 0;

function increase() {
  return ++count;
}

increase(); // count: 1
increase(); // count: 2
```

외부 변수를 직접 바꿈 → 부수 효과가 생김

순수 함수

객체를 바꿔도 비순수가 됨

```
// 비순수 . 원본 변경
```

```
function changeUser(user) {  
  user.name = "Kim";  
}
```

```
// 순수 . 새 객체 반환
```

```
function changeUser(user) {  
  return { ...user, name: "Kim" };  
}
```


가능하면 순수 함수를 쓰자

순수 함수는 동작을 예측하기 쉽고, 비순수 함수는 꼭 필요할 때만 쓰는 것이 바람직함

감사합니다.

JavaScript · 함수